OBI

Open Bus Interface

Protocol Reference — Signals · Waveforms · Read & Write Operations · Byte Enables

1 Chapter · Signal Reference Table · Byte Enable Table

Open Bus Interface (OBI)

Description

The Open Bus Interface (OBI) is a lightweight bus protocol used to connect masters and slaves in embedded and SoC designs. It is designed to be easy to implement in RTL while still supporting practical features such as reads, writes, wait states, and byte enables.

At its core, OBI separates a transaction into two parts:

- **Request acceptance** using req and gnt
- **Transaction completion** using rvalid and, for reads, rdata

That split makes OBI simple to reason about:

- The master asks for a transfer
- The slave accepts it when ready
- The slave later returns the completion

This chapter focuses on the common minimal OBI-style interface shown in the waveform below.

Why OBI is Useful

- **Simple handshake:** easy to implement and verify
- **Low overhead:** good fit for small and medium interconnects
- **Configurable widths:** address and data widths are implementation-defined
- **Byte enables:** supports partial-word writes
- **Latency tolerant:** the slave can delay either acceptance or completion

Protocol Overview

An OBI transfer is considered **accepted** in the cycle where both req and gnt are high.

- **req** is driven by the master
- **gnt** is driven by the slave
- **rvalid** indicates that the slave is returning the response
- **rdata** is meaningful only for read responses

In this simplified interface, the protocol is typically used with **one outstanding transaction at a time**. That means the master waits for rvalid before starting the next transfer.

Signals

The table below lists all interface signals, their direction, and their function.

Signal	Direction	Description
Clock and Reset		
`clk`	Input (global)	System clock. All interface signals are sampled on the rising edge.
`arst_n`	Input (global)	Asynchronous active-low reset. When low, the interface

Signal	Direction	Description
		returns to its idle state.
Request Channel — Master to Slave		
`req`	Master → Slave	Request valid. The master asserts this when it has a valid transfer to initiate.
`addr`	Master → Slave	Transfer address. Must remain stable while waiting for gnt.
`we`	Master → Slave	Write enable. 1 = write, 0 = read.
`wdata`	Master → Slave	Write data. Valid only for write transactions.
`be`	Master → Slave	Byte enable mask. Selects which byte lanes are active during a write.
Response / Handshake Channel — Slave to Master		
`gnt`	Slave → Master	Grant. The slave asserts this to accept the current request.
`rvalid`	Slave → Master	Response valid. Indicates the transaction has completed.
`rdata`	Slave → Master	Read data. Valid only when rvalid = 1 and the transfer was a read.

Transfer Rules

- The master asserts **req** when it has a valid request.
- While waiting for **gnt**, the master must keep all request signals (addr, we, wdata, be) stable.
- A request is **accepted** only in the cycle where req && gnt is true.
- The response may come later, indicated by **rvalid**.
- For writes, **rvalid** is a completion acknowledgement; **rdata** is ignored.
- For reads, the master samples **rdata** when **rvalid** is asserted.

Waveform

How to Read the Waveform

- The master raises **req** together with address and control information.
- If the slave is not ready, **gnt** stays low and the request must remain unchanged.
- When **gnt** goes high, the request is accepted on that rising edge.
- Some cycles later, the slave asserts **rvalid** to finish the transfer.
- If the transfer was a read, **rdata** is valid in the **rvalid** cycle.

Operations

Reset

When arst_n is low, the interface returns to its idle state.

Master Side

- **req** = 0

- **addr**, **we**, **wdata**, and **be** are don't-care unless the design specifies otherwise

Slave Side

- **gnt** = 0
- **rvalid** = 0
- **rdata** is don't-care unless the design specifies otherwise

After reset is released, transfers can begin on a subsequent rising edge of clk.

Read Operation

For a read transfer, the master:

- Drives the target address on **addr**
- Sets **we** = 0
- Asserts **req** = 1
- Keeps the request stable until the slave asserts **gnt**

Acceptance Phase

- The slave may immediately assert **gnt**, or it may insert wait states.
- The read request is accepted on the cycle where req && gnt is true.

Completion Phase

- At a later cycle, the slave asserts **rvalid** = 1.
- In that same cycle, **rdata** contains valid read data.
- The master samples **rdata** on the rising edge when **rvalid** = 1.

Write Operation

For a write transfer, the master:

- Drives the target address on **addr**
- Sets **we** = 1
- Drives write data on **wdata**
- Drives the byte mask on **be**
- Asserts **req** = 1
- Keeps all request signals stable until **gnt**

Acceptance Phase

- The write is accepted when req && gnt is true.
- At that point, the slave has accepted the address, control, and write data.

Completion Phase

- The slave later asserts **rvalid** = 1 to indicate completion.
- During a write response, **rdata** is not used and should be ignored.

Byte Enable Examples

For a 32-bit data bus, be is typically 4 bits wide. The table below shows common byte enable patterns:

be value	Effect
`4'b1111`	Writes all 4 bytes (full word)
`4'b0001`	Writes only byte 0 (LSB)
`4'b0011`	Writes the lower halfword (bytes 0–1)
`4'b1100`	Writes the upper halfword (bytes 2–3)

Practical Notes

- OBI allows the request and response phases to be separated by any number of cycles.
- A slave can throttle requests by keeping **gnt** low.
- A slave can add response latency by delaying **rvalid**.
- In this simplified usage model, responses return in order because only one request is outstanding.

> 💡 If you remember only one rule: **keep the request stable until grant, then wait for rvalid to know the transfer is finished.**